

Week 1 - Day 11a Supplementary: Time Complexity

This notebook contains some exercises related to time complexity.

In each of the exercises, your task is to write a function that solves the problem. The function `check_encrypted` will print a readable message if you got the right answer, and otherwise it will print some strange characters.

```
# This function is for testing, do not modify
def check_encrypted(answer, secret):
    answer = str(answer)
    result = []
    offset = sum([ord(c) for c in answer])
    for i in range(len(secret)):
        result.append(chr(((offset + ord(answer[i%len(answer)])) +
ord(secret[i]) - 64)%224 + 32))
    print("".join(result))
```

Exercise 1: Largest Difference []

You are given a list of positive integers $a_0, a_1, a_2 \dots a_{n-1}$. Your task is to find the largest difference between two of the numbers. In other words, you should find the maximum possible value of $a_i - a_j$.

In the first test case, $n=1000$. This means that an $O(n^2)$ solution will run quite fast. For the second test case you will need a faster solution, for example $O(n)$. If you solve the first test case you will get a hint on how to solve the second!

Example: if `a = [5, 4, 12, 5]`, then the answer is 8 because $12 - 4 = 8$ is the largest difference.

```
# TODO: Implement this function to find the largest difference
def largest_difference(a):
    pass

# Test cases below. Do not modify
first_case = [pow(13247, i, 10**9+7) for i in range(1, 1001)]
check_encrypted(largest_difference(first_case),
"ø).*.Ä7@2Ä/:4(&5<Ä\"3*Ç,65=+&49Ç'1$;7à")

second_case = [pow(5743143, i, 10**9+7) for i in range(1, 10**5+1)]
check_encrypted(largest_difference(second_case), "+ÿ6¾!5'Ä- ',¾-
&Ç&3501#¿")
```

Exercise 2: Thunderstorm []

Tamiru did not get much sleep last night, because of the thunderstorms. Every time there was a lightning strike, he woke up. Now he wants to know the length of the longest section of unbroken sleep.

You are given a string s consisting of zeros and ones. If $s_i = 1$ it means that there was a lightning strike at second i . The task is to find the longest contiguous section of zeros in this string.

Example: If $s = 011000100$, then the answer is 3, because the longest section of zeros is 3.

Below you can find a solution to this problem. But the time complexity of this solution is $O(n^2)$, which means that it will be very slow when $n = 3 \cdot 10^5$. Rewrite the function `longest_sleep` below so that it runs fast enough.

```
# TODO: Implement this function to find Minase's longest time asleep
def longest_sleep(s):
    # Slow solution below.
    ans = 0
    for i in range(len(s)):
        j = i
        while j < len(s) and s[j] == '0':
            j += 1
        ans = max(ans, j-i)
    return ans

# Test case below. Do not modify
n = 3*10**5
s = ['0']*n
for i in range(n):
    x = pow(123713, i+1, n//2)
    if i%5000 == 0:
        x += n//2
    s[x] = '1'

check_encrypted(longest_sleep("".join(s)), "ÈàØÓãìÙßÚ")
```

Exercise 3: Fast Exponentiation []

When calculating $m^n \bmod k$, one method is to multiply m with itself n times. The time complexity of that is $O(n)$. But there is a faster way. If n is a power of two, so $n = 2^a$, then we can instead do like this:

$$m \cdot m = m^2$$

$$m^2 \cdot m^2 = m^4$$

$$m^4 \cdot m^4 = m^8$$

...

